Simulação e Teste de Software

Aula 8: Critérios de Cobertura Estrutural

Prof. Luciano Rossi

Departamento de Ciência da Computação
Centro Universitário FEI

2º Semestre de 2026

Roteiro da Disciplina

- Fundamentos e Taxonomia de Testes de Software
- Ciclo de Vida de Teste de Software (STLC)
- Planejamento Mestre de Testes e Análise de Risco
- Introdução à Automação de Testes com pytest
- Teste de Unidade Avançado (FIRST, padrão AAA)
- Técnicas Caixa-Preta: Partição de Equivalência e Valor-Limite
- Técnicas Caixa-Preta: Teste Combinatorial e *Property-Based*
- **Técnicas Caixa-Branca: Critérios de Cobertura Estrutural**
- Testes de Mutação e Avaliação da Eficácia de Suítes de Teste
- Test-Driven Development (TDD)
- Teste de Integração, Test Doubles e Níveis de Teste
- Teste Orientado a Objetos e de Componentes

Roteiro de Hoje

- Nas aulas 6 e 7, os casos de teste vieram da especificação; hoje eles vêm do código
- Grafo de fluxo de controle e complexidade ciclomática
- Cobertura de comandos e de ramos, medida com `coverage.py`
- Cobertura de condições e MC/DC
- Teste de laços e fluxo de dados
- Comparação entre os critérios

Da Especificação ao Código



Dois Testes Bastam?

Contexto

Regra do *login*: `autenticar` devolve `True` só se o usuário existe e a senha confere.

autenticacao.py

```python
USUARIOS = {"ana": "s3nh4", "bruno": "abc123"}
MODO_TESTE = True # ligado para depurar e esquecido

def autenticar(usuario, senha):
    if usuario not in USUARIOS:
        return False
    if MODO_TESTE:
        return True
    return USUARIOS[usuario] == senha
```

Os dois testes passam: `autenticar("ana", "s3nh4")` espera `True`. Já `autenticar("carlos", "x")` espera `False`.

Testes Passando, Porta Aberta

Exemplo

- Não bastam: `autenticar("ana", "errada")` também dá `True`, e qualquer senha entra
- A linha 9, único comando que confere a senha, nunca executa: com `MODO_TESTE` ligado, a função sempre retorna antes

Relatório do `coverage.py`, ferramenta vista no bloco 5

```
Name                Stmts   Miss  Cover   Missing
-------------------------------------------------
autenticacao.py         8      1    88%   9
```

A ideia da aula: olhar o código para achar o que os testes não exercitaram. O relatório aponta a linha 9 sem que ninguém precise imaginar o teste da senha errada.

Teste Caixa-Branca

Definição

Termos do encontro

- **Teste caixa-branca** (ou estrutural): família de técnicas de projeto de testes. Usa a estrutura interna do código, não só a especificação
- **Critério de cobertura**: regra que diz quais elementos do código os testes devem exercitar. Exemplos: comandos, ramos, condições
- **Cobertura**: percentual desses elementos que a suíte de fato exercitou

De volta ao *login*

Partição e valor-limite (aula 6) partem da especificação e não enxergam o `MODO_TESTE`. A cobertura de comandos aponta o comando que nenhum teste executou.

Caixa-Branca em Cinco Perguntas

Conceito

- **O quê**: família de técnicas de teste. Os critérios de cobertura são as regras; ferramentas como `coverage.py` medem o cumprimento
- **Por quê**: revela código que nenhum teste executa. Exemplos: ramos esquecidos, casos especiais, código morto[1]
- **Quando e onde**: principalmente no teste de unidade, por quem tem acesso ao código. Não acha funcionalidade que falta: código inexistente não aparece na cobertura
- **Quem**: o desenvolvedor, na automação do dia a dia. Também equipes de software crítico, cujas normas exigem critérios
- **Como**: modela o código como grafo e escolhe um critério. Depois, projeta testes até cobrir o que o critério exige

[1] *Código morto: trecho que nenhuma execução alcança.*

Grafo de Fluxo de Controle



Do Código ao Grafo

Definição

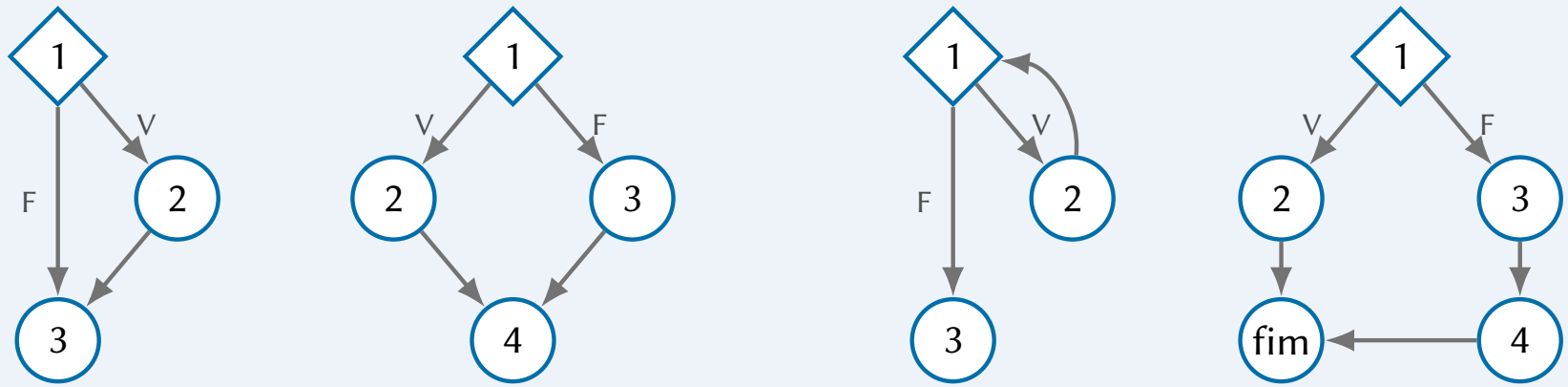
Grafo de fluxo de controle (GFC)

- **Comando**: instrução executável do Python, como atribuição, `if` ou `return`. Em geral, ocupa uma linha
- **GFC**: grafo orientado do código. Cada nó é um comando; cada aresta, uma passagem possível de um nó a outro
- **Nó de decisão**: nó de um `if`, `elif`, `while` ou `for`; nos grafos, é o losango
- **Ramo**: aresta que sai de um nó de decisão, rotulada V ou F. Todo ramo é aresta, mas nem toda aresta é ramo
- **Entrada e saída**: nós onde toda execução começa e termina. Vários `return` levam ao mesmo nó de saída

Cada nó recebe o número da linha do seu comando. Comandos em sequência sem desvio formam um **bloco básico**, que pode virar um nó só.

As Estruturas no Grafo

Diagrama



Fonte: criada pelo autor.

- No laço, o nó 1 testa se há mais uma volta; a aresta de 2 para 1 é a volta
- No `for` do Python, a pergunta é se ainda há elemento a percorrer
- Com `return` no meio, o ramo V pula o resto da função e vai direto ao nó *fim*

Tarifa de Estacionamento

centro universitário FEI

Código

Especificação: mensalista não paga. Avulso paga R$10 na primeira hora e R$5 por hora adicional, até R$50 por dia.

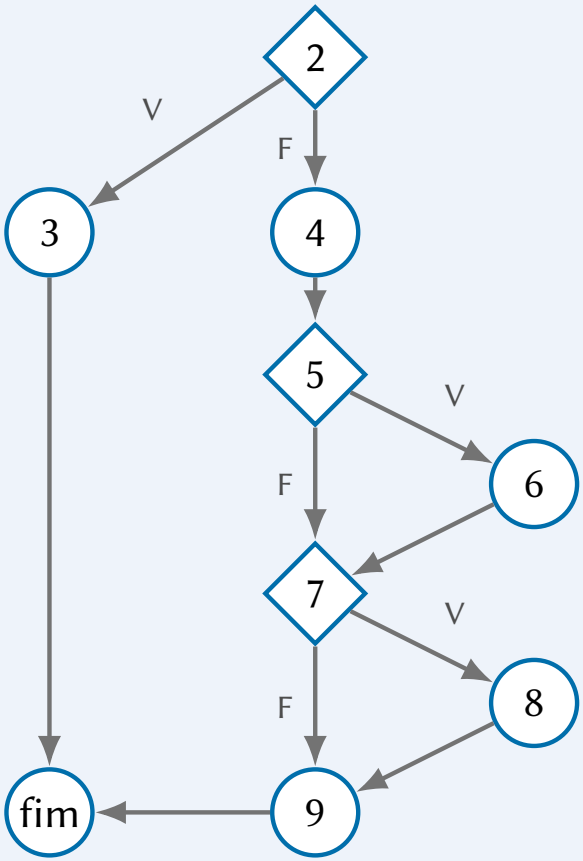
estacionamento.py

```python
def tarifa(horas, mensalista):
    if mensalista:
        return 0
    valor = 10
    if horas > 1:
        valor = valor + (horas - 1) * 5
    if valor > 50:
        valor = 50
    return valor
```

Esta função acompanha a aula inteira.

O Grafo da Tarifa

Diagrama



Fonte: criada pelo autor.

estacionamento.py

```
1  def tarifa(horas, mensalista):
2      if mensalista:
3          return 0
4      valor = 10
5      if horas > 1:
6          valor = valor + (horas - 1) * 5
7      if valor > 50:
8          valor = 50
9      return valor
```

9 nós, 11 arestas e 3 nós de decisão[1].

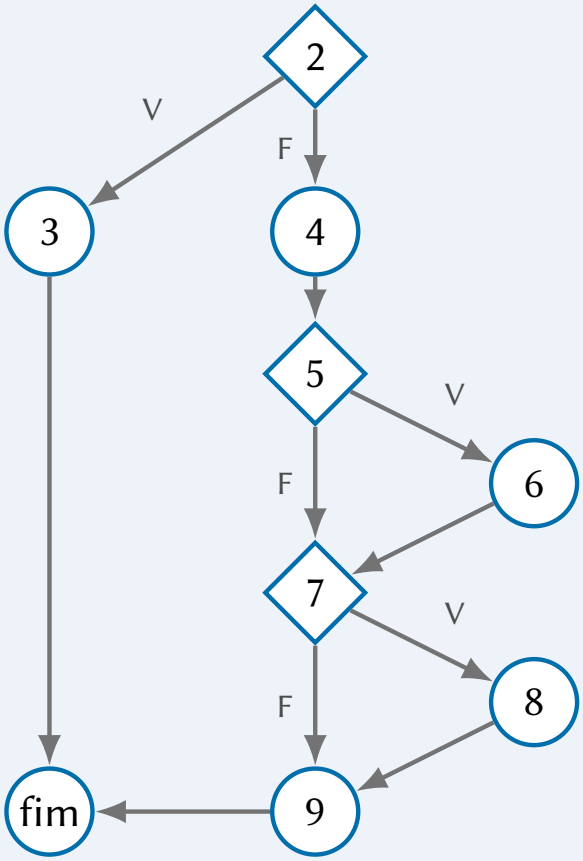
[1] *4 e 5 formam um bloco básico e poderiam ser um nó só. Separados, o grafo segue o código, e $V(G)$ não muda.*

Complexidade Ciclomática e Caminhos



Complexidade Ciclomática

Definição



Fonte: criada pelo autor.

Complexidade ciclomática $V(G)$

Métrica de software calculada sobre o GFC G. Com uma entrada e uma saída:

$$V(G) = E - N + 2 \qquad V(G) = P + 1$$

E: arestas. N: nós. P: nós de decisão com dois ramos. O significado, o tamanho de uma base de caminhos, vem a seguir.

Na tarifa

$V(G) = 11 - 9 + 2 = 4$, ou $V(G) = 3 + 1 = 4$.

$V(G)$ em Cinco Perguntas

Conceito

- **O quê**: métrica, um número calculado sobre o GFC; não é critério de cobertura nem ferramenta
- **Quem**: proposta por Thomas McCabe em 1976[1]. Ferramentas como radon (Python) e SonarQube a calculam
- **Por quê**: dá o tamanho de uma base de caminhos, conjunto definido nos próximos slides que cobre todos os ramos
- **Quando e onde**: por função ou método. Valor alto indica função difícil de testar, candidata a ser dividida
- **Como**: conte arestas e nós, ou conte as decisões e some 1

Com condição composta (`a and b`), cada condição costuma contar como uma decisão. O `and` também desvia a execução.

[1] *T. J. McCabe, A Complexity Measure, IEEE Transactions on Software Engineering, 1976.*

Caminho como Vetor

Definição

Caminhos e vetores

- **Caminho**: sequência de nós da entrada até a saída do GFC
- **Vetor do caminho**: uma posição por aresta do grafo, com o número de vezes que o caminho passa por ela[1]
- **Combinação linear** de um conjunto de vetores: soma desses vetores, cada um multiplicado por um número real
- Os números podem ser zero, negativos ou frações; no exemplo adiante, $G4 = G2 + G3 - G1$

Com caminhos como vetores, "o caminho é novo?" vira pergunta de álgebra. Ele pode ser montado a partir dos outros?

[1] *A. H. Watson e T. J. McCabe, Structured Testing: A Testing Methodology Using the Cyclomatic Complexity Metric, NIST Special Publication 500-235, 1996, seção 2.3.*

Caminhos Linearmente Independentes

centro universitário FEI

Definição

Independência e base

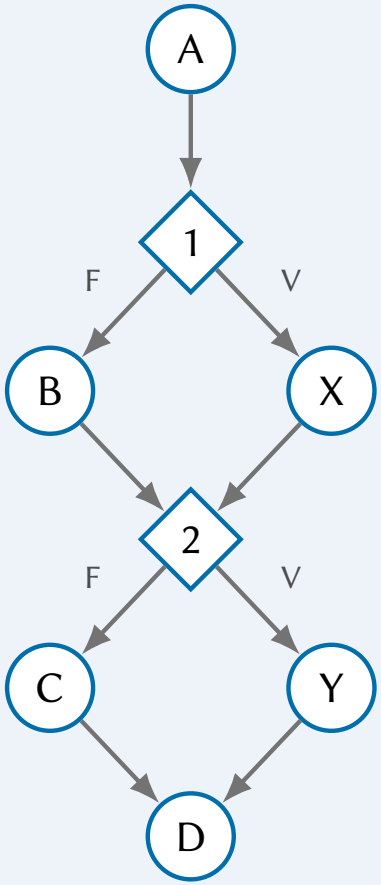
- **Caminho linearmente independente** de um conjunto: seu vetor não é combinação linear dos vetores dele
- **Base de caminhos**: conjunto de caminhos independentes. Todo caminho do grafo é combinação linear deles
- Toda base de um GFC tem exatamente $V(G)$ caminhos, embora existam muitas bases diferentes
- **Caminho inviável**: caminho do grafo que nenhuma entrada consegue percorrer

Teste de caminho básico: técnica que escolhe um caso de teste para cada caminho de uma base[1].

[1] *Técnica proposta por McCabe; descrita em R. S. Pressman, Software Engineering: A Practitioner's Approach, 5ª ed., 2001, seção 17.4.*

Duas Decisões em Sequência

Exemplo



Fonte: criada pelo autor.

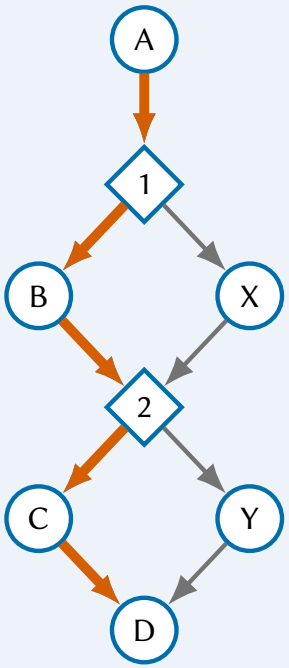
exemplo.py

```python
def exemplo(c1, c2):
    print("A")
    if c1: # no 1
        print("X")
    else:
        print("B")
    if c2: # no 2
        print("Y")
    else:
        print("C")
    print("D")
```

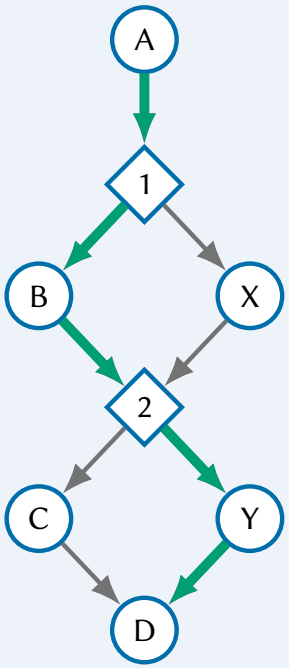
8 nós e 9 arestas: $V(G) = 9 - 8 + 2 = 3$.

Os Quatro Caminhos

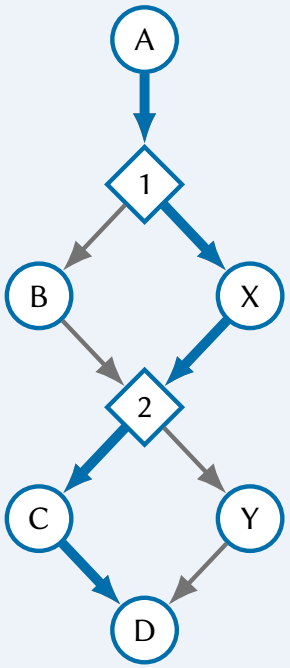
Exemplo



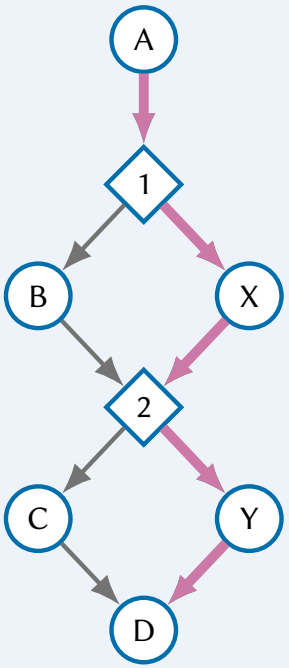
G1: A, 1, B, 2, C, D



G2: A, 1, B, 2, Y, D



G3: A, 1, X, 2, C, D



G4: A, 1, X, 2, Y, D

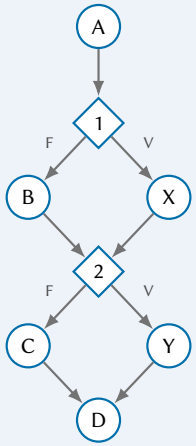
Fonte: criada pelo autor.

Quatro caminhos viáveis, um por combinação de c1 e c2. Mas $V(G) = 3$: um deles depende dos outros três.

Os Caminhos como Vetores

Exemplo

	A→1	1→B	1→X	B→2	X→2	2→C	2→Y	C→D	Y→D
G1	1	1	0	1	0	1	0	1	0
G2	1	1	0	1	0	0	1	0	1
G3	1	0	1	0	1	1	0	1	0
G4	1	0	1	0	1	0	1	0	1



Fonte: criada pelo autor.

Uma combinação linear

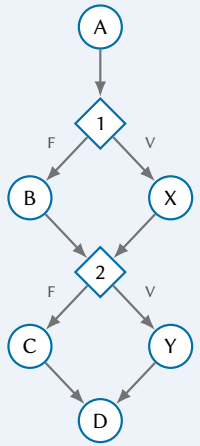
G4 = G2 + G3 − G1, posição por posição. Na coluna 1→X: $1 = 0 + 1 - 0$; na coluna 2→C: $0 = 0 + 1 - 1$.

Qualquer um dos quatro se escreve com os outros três. Qualquer trio é uma base.

Aresta Nova Não É o Critério

Conceito

Regra prática comum: aceitar na base o caminho que passa por uma aresta ainda não usada. O resultado depende da ordem.



Fonte: criada pelo autor.

Ordem G1, G2, G3, G4

- G1: todas as suas arestas são novas
- G2: 2→Y e Y→D
- G3: 1→X e X→2
- G4: nenhuma; base com 3 caminhos

Ordem G2, G3, G1, G4

- G2: todas as suas arestas são novas
- G3: 1→X, X→2, 2→C e C→D
- G1 e G4: nenhuma; a regra para em 2

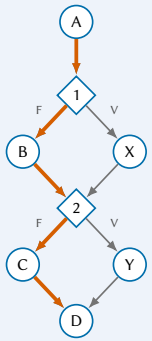
G1 não traz aresta nova, mas é independente de G2 e G3. O critério é a independência linear, não a aresta nova.

Aresta Nova como Atalho

Conceito

Critério simplificador da aresta nova

- **Regra**: aceite o caminho que percorre uma aresta ainda não usada na base
- **Por que vale**: nessa aresta, os aceitos têm 0 e o novo tem 1; nada gera esse 1
- **Limite**: suficiente, mas não necessária; na ordem errada, para antes de $V(G)$



Fonte: criada pelo autor.

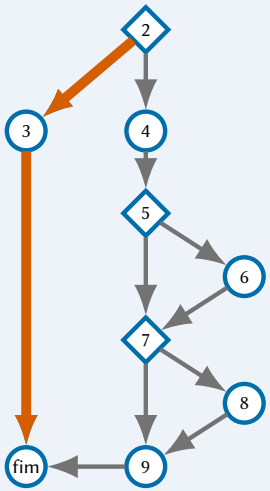
Uma ordem que não trava

Parta de um **caminho típico**, a referência: o que a execução segue no uso comum, sem erro nem exceção. Inverta uma decisão por vez e volte à referência[1]; cada inversão usa uma aresta nova. No exemplo: G1 (laranja), G3 (nó 1) e G2 (nó 2).

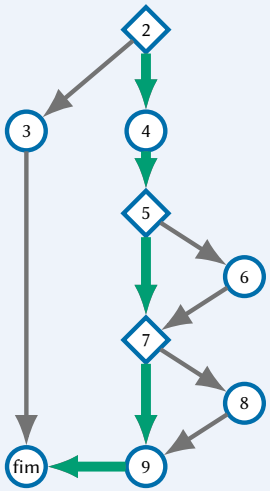
[1] *Método do caminho de referência: A. H. Watson e T. J. McCabe, NIST Special Publication 500-235, 1996, seção 6.*

Base de Caminhos da Tarifa

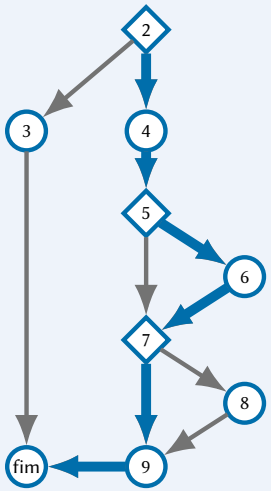
Exemplo



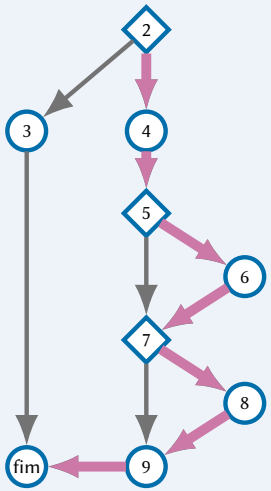
P1



P2



P3



P4

Fonte: criada pelo autor.

Caminho	Como foi obtido, na ordem	Entrada	Esperado
P3	referência: uma permanência comum	`(3, False)`	20
P1	inverte o nó 2; aresta nova 2→3	`(3, True)`	0
P2	inverte o nó 5; aresta nova 5→7	`(1, False)`	10
P4	inverte o nó 7; aresta nova 7→8	`(20, False)`	50

P4 não serve de referência: inverter o nó 5 cai em um caminho inviável[1].

[1] *O caminho 2, 4, 5, 7, 8, 9, fim: com até 1 hora, o valor é 10 e nunca passa de 50.*

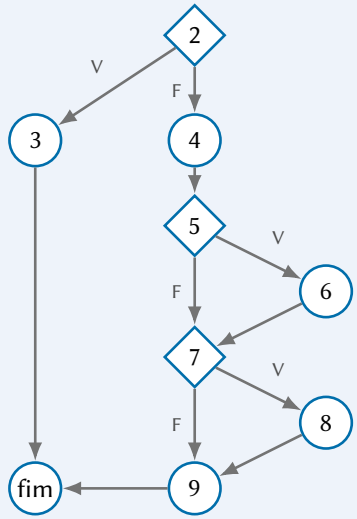
Quantos Testes Bastam?

Pausa ativa

Com a tarifa e os quatro caminhos da base:

- Qual é o menor conjunto de testes que executa todo comando da função ao menos uma vez?
- E o menor que faz cada decisão assumir V e F?

Discuta com a dupla e escreva as entradas antes do próximo slide.



Fonte: criada pelo autor.

Comandos e Ramos

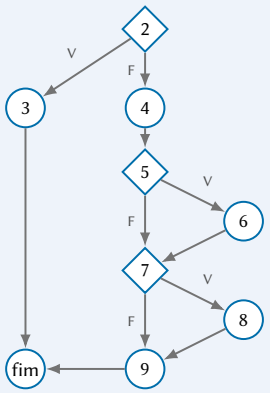


Comandos e Ramos

Definição

Dois critérios de fluxo de controle

- **Cobertura de comandos** (C0): critério que exige executar todo comando ao menos uma vez; no GFC, visitar todo nó. As ferramentas chamam de cobertura de linhas
- **Cobertura de ramos** (C1, ou de decisões): toda decisão assume V e F. No GFC, percorrer todo ramo e, com isso, toda aresta



Fonte: criada pelo autor.

Na tarifa

- C0: P1 e P4, 2 testes. P4 executa todos os comandos menos o `return 0`
- C1: P1, P2 e P4, 3 testes. P2 traz os ramos F dos dois últimos `if`
- Base de caminhos: 4 testes

$V(G)$ é um Teto

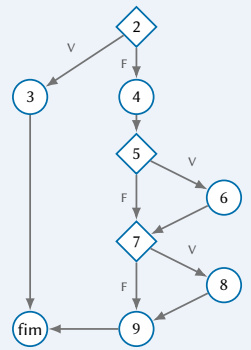
Conceito

Quantidade de testes por critério

$$\text{testes para C0} \leq \text{testes para C1} \leq V(G)$$

A base de caminhos sempre cobre todos os ramos. Mas um conjunto menor pode bastar: na tarifa, C1 pede 3 testes e $V(G) = 4$.

- C1 inclui C0: quem percorre todo ramo visita todo nó
- O contrário não vale: o próximo slide mostra um defeito que C0 deixa passar



Fonte: criada pelo autor.

Quando Comandos Não Bastam

Código

cupom.py

```
1  from estacionamento import tarifa
2
3  def tarifa_com_cupom(horas, cupom):
4      if cupom:
5          desconto = 5
6      return tarifa(horas, False) - desconto
```

- O teste `assert tarifa_com_cupom(3, True) == 15` passa e executa todo comando: 100% de C0
- Sem cupom, a linha 6 lança `UnboundLocalError`, porque `desconto` não existe
- Só o ramo F do `if`, exigido por C1, revela o defeito

Medindo Cobertura



`coverage.py` em Cinco Perguntas

Conceito

- **O quê**: ferramenta de terminal e biblioteca Python que mede cobertura. Não é critério nem *framework* de teste
- **Quem**: escrita por Gareth Rees em 2001, mantida por Ned Batchelder desde 2004. O pytest-cov, *plugin* do pytest, a usa por baixo
- **Por quê**: mostra quais comandos e quais ramos a suíte nunca executou
- **Como**: registra, durante os testes, cada comando executado e cada passagem de um comando a outro
- **O que mede**: comandos, por padrão; ramos, com `--branch`. Não mede condições nem MC/DC[1], que veremos no bloco 6

[1] *MC/DC (Modified Condition/Decision Coverage): cobertura de condição e decisão modificada; cada condição precisa mostrar, sozinha, que muda o resultado da decisão.*

Medindo com coverage.py

centro universitário FEI

Código

Terminal

```
pip install coverage
coverage run --branch --source=cupom -m pytest
coverage report -m
```

Saída do `coverage report -m`

```
Name       Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------
cupom.py       5      0      2      1    86%   4->6
```

Sem `--branch`, o mesmo relatório mostra 100%. `--source` limita o relatório ao módulo testado, e `coverage html` gera páginas com as linhas coloridas.

Lendo o Relatório

Conceito

O relatório do slide anterior, coluna a coluna. O único teste é `tarifa_com_cupom(3, True)`.

Coluna	Valor	Leitura no `cupom.py`
`Stmts`	5	comandos do arquivo: linhas 1, 3, 4, 5 e 6 (o `def` conta)
`Miss`	0	comandos nunca executados: nenhum, 100% de C0
`Branch`	2	ramos do `if` da linha 4: 4→5 (V) e 4→6 (F)
`BrPart`	1	decisões que seguiram só um ramo: o `if`, sempre V
`Missing`	`4->6`	o que faltou: o ramo F, da linha 4 direto para a 6
`Cover`	86%	executados sobre o total: $(5+1)/(5+2)$

Todo comando ou ramo descoberto pede uma decisão

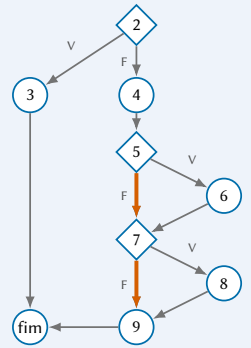
Falta um teste, o código está morto e deve sair, ou a situação é difícil de provocar e fica registrada com a justificativa.

Cobertura com pytest-cov

Código

Terminal

```
pip install pytest-cov
pytest --cov=estacionamento --cov-branch \
       --cov-report=term-missing
```



Fonte: criada pelo autor.

Saída, com os testes P1 e P4 (C0 completa)

```
Name                 Stmts   Miss Branch BrPart  Cover   Missing
----------------------------------------------------------------
estacionamento.py        9      0      6      2    87%   5->7, 7->9
```

Faltam os ramos F dos `if` das linhas 5 e 7, em laranja no grafo. P2, `tarifa(1, False)`, traz os dois: com ele, 100%.

Condições



Condição e Decisão

Definição

isencao.py

```
1 def isento(mensalista, convenio, horas):
2     if mensalista or (convenio and horas <= 2):
3         return True
4     return False
```

Dois termos

- **Condição**: expressão booleana sem `and`, `or` ou `not`. Aqui: `mensalista`, `convenio` e `horas <= 2`
- **Decisão**: a expressão inteira que escolhe o ramo

100% de Ramos, Duas Condições Paradas

Código

Suíte com dois testes:

- `isento(True, False, 5)`, esperado `True`
- `isento(False, False, 5)`, esperado `False`

Saída do `coverage report -m`

```
Name          Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------------
isencao.py        4      0      2      0   100%
```

`convenio` e `horas <= 2` nunca foram verdadeiros: a regra do convênio nunca foi testada.

Critérios de Condição

Definição

Critérios que olham dentro da decisão

- **Cobertura de condições** (CC): toda condição assume V e F ao menos uma vez. Não garante que a decisão assuma V e F
- **Condição/decisão** (CDC): CC e C1 juntas
- **Condições múltiplas** (MCC): toda combinação de valores das condições; com n condições, 2^n testes

CC sem C1, com `a and b`

Os testes $(a, b) = (\mathrm{V}, \mathrm{F})$ e (F, V) fazem cada condição assumir V e F, mas a decisão assume F nos dois: o ramo V nunca é percorrido.

MC/DC

Definição

Condição/decisão modificada (MC/DC)

Critério que exige CDC e o efeito independente de cada condição. Para cada condição, dois testes que diferem só nela e mudam o valor da decisão.

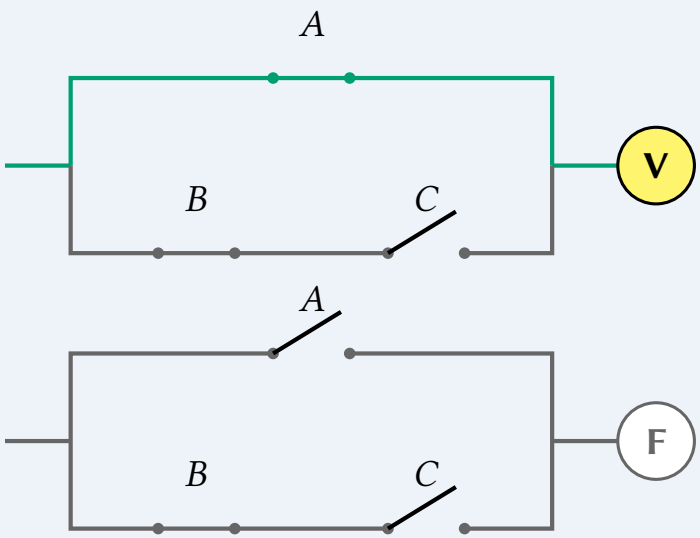
- **Por quê**: prova que cada condição importa, com cerca de $n + 1$ testes no lugar dos 2^n do MCC
- **Quem**: definido na aviação, na norma DO-178B (1992), e mantido na DO-178C (2011). É exigido no software de nível A, cuja falha pode ser catastrófica
- **Quando**: decisões compostas em software crítico; em código comum, costuma bastar C1

Efeito Independente como Circuito

Conceito

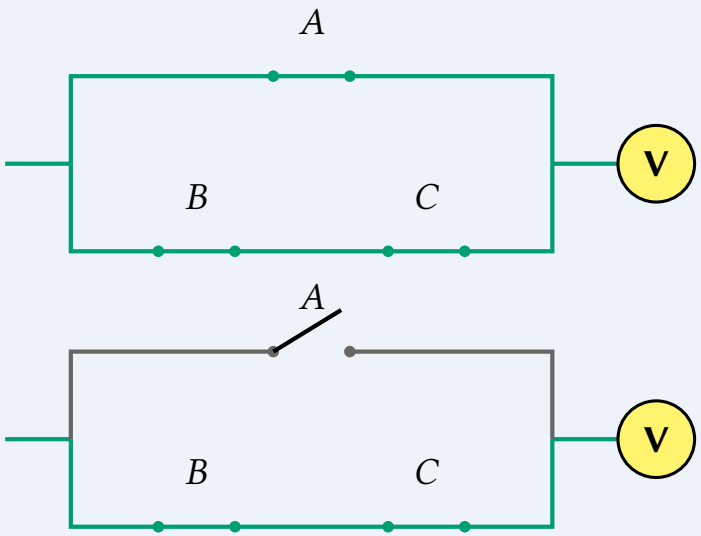
A or (B and C): A em paralelo com B e C em série. Chave fechada é V; lâmpada acesa é decisão V.

Só A muda: a lâmpada apaga ✔



A decide sozinha: é um **par de** A

Só A muda: a lâmpada continua ✘

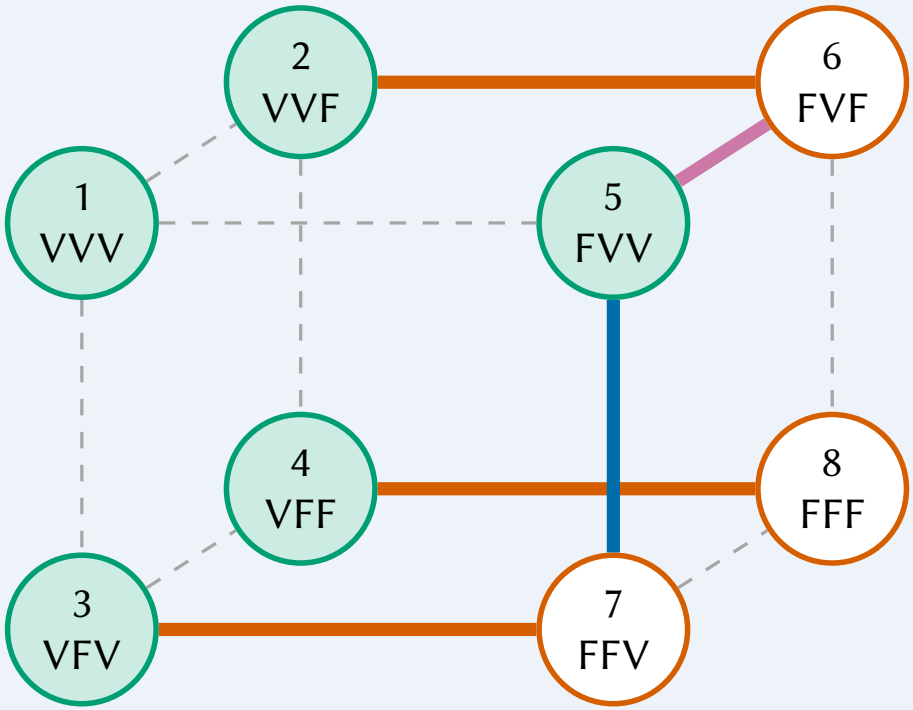


B e C *mascaram* A: não é par

MC/DC pede, para cada condição, um par como o da esquerda.

MC/DC Passo a Passo: os Pares

Exemplo



Fonte: criada pelo autor.

- Cada bola é uma das 8 combinações de ABC: **verde** se a decisão é V, branca se é F
- Cada aresta troca uma condição: **horizontal** A, **vertical** B, **diagonal** C
- **Par**: aresta grossa, de verde a branca. As tracejadas não mudam a decisão

$A =$ `mensalista`, $B =$ `convenio`, $C =$ `horas <= 2`. A tem três pares; B e C, um cada, ambos na combinação 5.

MC/DC Passo a Passo: o Conjunto

Exemplo

B e C têm um par cada e obrigam 5, 6 e 7; para A, o par 3–7 reaproveita a 7.

Comb.	Teste	Esperado	Par de
3	`isento(True, False, 1)`	V	A
5	`isento(False, True, 1)`	V	B e C
6	`isento(False, True, 5)`	F	C
7	`isento(False, False, 1)`	F	A e B

Impacto: o que a suíte passa a detectar

Código que ignore `mensalista` (par 3–7), `convenio` ou troque `and` por `or` (5–7), ou ignore `horas <= 2` (5–6). São 4 testes no lugar dos 8 do MCC; os 2 que davam 100% de ramos não pegavam os defeitos em `convenio` e `horas`.

Curto-Circuito em Python

Conceito

Avaliação em curto-circuito

`or` para de avaliar no primeiro operando verdadeiro e `and`, no primeiro falso: o resultado já está decidido.

- Nas combinações 1 a 4 do cubo, A é V e B e C nem chegam a ser avaliadas
- O par 3–7 vale por si: só A muda. A variante por *mascaramento* aceita também pares em que B ou C mudam sem ser avaliadas, como 1–7
- O `coverage.py` trata a linha do `if` como uma decisão com dois ramos. As condições ficam invisíveis no relatório
- MC/DC em Python se confere à mão, com o cubo, ou com ferramenta especializada

Teste de Laços



Teste de Laços

Definição

C1 se satisfaz com zero e uma volta, mas os defeitos de laço aparecem na última volta ou além dela.

Teste de laços

Técnica caixa-branca que testa a validade dos laços. Para um laço simples com no máximo n voltas[1]:

- Pular o laço: zero voltas
- Uma volta e duas voltas
- m voltas, um valor típico com $m < n$
- $n - 1$, n e $n + 1$ voltas

Os testes de laço são valores-limite (aula 6) aplicados ao número de voltas.

[1] *B. Beizer, Software Testing Techniques, 2ª ed., 1990, apresentado em R. S. Pressman, Software Engineering: A Practitioner's Approach, 5ª ed., 2001, seção 17.5.3.*

Laços no Total do Mês

Exemplo

mensal.py

```
1  def total_do_mes(permanencias):
2      total = 0
3      for horas in permanencias:
4          total = total + tarifa(horas, False)
5      if total > 300:
6          total = 300
7      return total
```

Casos de laço

Zero voltas: `[]` dá 0. Uma: `[1]` dá 10. Duas: `[1, 3]` dá 30. Típico: dez entradas de 3 horas dão 200. Sem número máximo de entradas, os testes $n - 1$, n e $n + 1$ não se aplicam; o teto de R$300 limita o valor, não as voltas.

Laços Aninhados

Conceito

Aplicar os testes de laço simples a cada laço aninhado multiplica os testes. O procedimento de Beizer reduz esse número:

1. Comece pelo laço mais interno, com todos os externos no mínimo de voltas
2. Aplique a ele os testes de laço simples
3. Suba um nível: teste esse laço com os externos no mínimo e os internos em valores típicos
4. Repita até o laço mais externo

Laços em sequência, um depois do outro, são testados como laços simples quando são independentes.

Fluxo de Dados



O que os Ramos Não Veem

Definição

C1 garante que cada ramo foi percorrido, mas não em que combinação. O valor definido em um comando pode nunca chegar, nos testes, ao comando que o usa.

Termos de fluxo de dados

- **Definição** (def): comando que atribui valor a uma variável
- **Uso computacional** (*c-use*): leitura da variável em um cálculo ou `return`
- **Uso em predicado** (*p-use*): leitura da variável na condição de um `if` ou laço
- **Par definição-uso** (par du): uma definição e um uso da mesma variável, ligados por um trecho de caminho sem outra definição dela[1]

[1] *Não confundir com o par de MC/DC: lá, o par é de testes; aqui, de comandos.*

Pares du na Cobrança

</> Código

cobranca.py

```
1  from estacionamento import tarifa
2
3  def cobrar(horas, convenio):
4      valor = tarifa(horas, False)
5      if convenio:
6          valor = valor - 3
7      if horas > 12:
8          valor = valor + 15
9      return valor
```

- Convênio desconta R$3; acima de 12 horas, soma pernoite de R$15
- `valor`: definições nas linhas 4, 6 e 8; usos nas linhas 6, 8 e 9
- `convenio` e `horas` têm *p-use* nas linhas 5 e 7

Os Seis Pares de `valor`

Exemplo

Par	Caminho exigido	Coberto por C1?	Exemplo
(4, 6)	convênio	sim	`(20, True)`
(4, 8)	sem convênio, mais de 12 horas	**não**	`(20, False)`
(4, 9)	sem convênio, até 12 horas	sim	`(1, False)`
(6, 8)	convênio, mais de 12 horas	sim	`(20, True)`
(6, 9)	convênio, até 12 horas	**não**	`(3, True)`
(8, 9)	mais de 12 horas	sim	`(20, True)`

A coluna C1 considera a suíte `(20, True)` e `(1, False)`, que já percorre todas as arestas.

Dois testes cobrem todos os ramos e deixam dois pares de fora. Nenhum leva o valor sem desconto ao pernoite, nem o com desconto direto ao `return`.

All-Defs e All-Uses

Definição

Critérios de fluxo de dados

- **All-Defs**: toda definição alcança ao menos um de seus usos; na cobrança, `(20, True)` sozinho já cumpre
- **All-Uses**: todo par du é percorrido; na cobrança, 4 testes: os dois da suíte C1, `(20, False)` e `(3, True)`

- **O quê e quem**: critérios de cobertura propostos por Sandra Rapps e Elaine Weyuker em 1985[1]
- **Quando**: funções em que o mesmo valor passa por várias transformações. Poucas ferramentas medem: aplica-se à mão nos trechos críticos

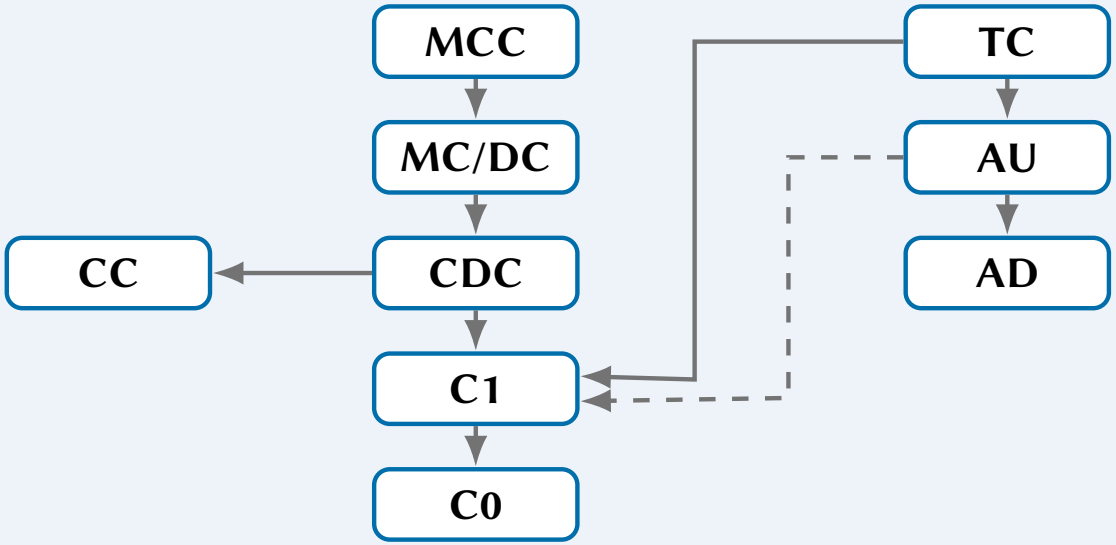
[1] *S. Rapps e E. J. Weyuker, Selecting Software Test Data Using Data Flow Information, IEEE Transactions on Software Engineering, 1985.*

Comparando os Critérios

Um Critério Inclui Outro

Diagrama

Inclusão: o critério A inclui o B quando todo conjunto de testes que satisfaz A também satisfaz B.



Fonte: criada pelo autor.

MCC	condições múltiplas
MC/DC	condição/decisão modificada
CDC	condição/decisão
C1	ramos
C0	comandos
CC	condições
TC	todos os caminhos
AU	All-Uses
AD	All-Defs

Seta cheia de A para B: A inclui B. Tracejada: vale se toda decisão usa uma variável. Fontes: Rapps e Weyuker (1985); DO-178B (1992).

CC não inclui C1, e TC não inclui CDC: o caminho não vê as condições.

Cobertura Não é Verificação

centro universitário FEI

Código

test_sem_assert.py

```python
from estacionamento import tarifa
def test_tarifas():
    tarifa(3, True); tarifa(1, False); tarifa(20, False)
```

Saída do `coverage report -m`

```
Name                  Stmts   Miss Branch BrPart  Cover   Missing
-----------------------------------------------------------------
estacionamento.py         9      0      6      0   100%
```

100% de ramos sem nenhum `assert`: a cobertura diz o que executou, não o que foi conferido. Aula 9: medir se os testes detectam defeitos.

Exercícios



Pedágio

Exercício 1

pedagio.py

```python
def pedagio(eixos, tag):
    if eixos <= 2:
        valor = 8
    elif eixos <= 4:
        valor = 15
    else:
        valor = 6 * eixos
    if tag:
        valor = valor * 0.95
    return valor
```

Desenhe o GFC e calcule $V(G)$ pelas duas fórmulas. Monte uma base de caminhos, com entrada e valor esperado de cada um.

Multa da Biblioteca

Exercício 2

biblioteca.py

```python
def multa_atraso(dias, renovou):
    multa = 0
    if dias > 0:
        multa = 2 * dias
    if renovou:
        multa = multa / 2
    return multa
```

a) Dê um conjunto mínimo de testes para C0 e um para C1

b) A suíte tem um só teste, `multa_atraso(5, True)`. Com `--branch`, o que o `coverage report -m` mostra em `Missing`?

c) Compare o número de testes de C1 com $V(G)$

Aprovação de Crédito

Exercício 3

credito.py

```python
def aprovar(renda, sem_divida, fiador):
    if renda >= 3000 and (sem_divida or fiador):
        return True
    return False
```

Chame as condições de A (`renda >= 3000`), B (`sem_divida`) e C (`fiador`).

a) Dê dois testes que satisfaçam CC sem satisfazer C1

b) Monte o cubo das oito combinações e encontre os pares de cada condição

c) Escolha um conjunto MC/DC mínimo e compare seu tamanho com o do MCC

Síntese

- O GFC transforma o código em grafo. $V(G)$ conta os caminhos da base e é um teto para os testes de ramos
- Comandos, ramos, condição/decisão e MC/DC: critérios cada vez mais exigentes. O `coverage.py` mede os dois primeiros
- O que os ramos não veem: o teste de laços olha as voltas; o fluxo de dados, o caminho de cada valor
- 100% de cobertura diz que o código executou, não que os testes conferem o resultado

Simulação e Teste de Software

Aula 8: Critérios de Cobertura Estrutural

Prof. Luciano Rossi

Departamento de Ciência da Computação
Centro Universitário FEI

2º Semestre de 2026